

Prof. Etienne Messerli

SysLog2 / CSN

Labo Système séquentiel simple : Timer

Contexte

Un Timer est un périphérique matériel permettant de mesurer des durées. Il se trouve généralement inclus dans toutes les architectures des microcontrôleurs modernes. Son rôle est de permettre la synchronisation des opérations que le microcontrôleur est chargé d'effectuer.

Le concept de temps dans un système informatique est typiquement réalisé par le comptage d'un certain nombre de périodes d'horloge afin de pouvoir maîtriser des événements temps réel. Sachant la fréquence de l'horloge utilisée, nous pouvons connaître précisément le temps écoulé. Cela est réalisé, dans un microcontrôleur, par l'utilisation de compteur disposant de différent mode de fonctionnement et de comparateur pour détecter différentes durées.

Le but d'inclure un Timer dans un microcontrôleur est de décharger le processeur pour de simples mesures de temps. En effet la solution « soft » est de programmer le comptage avec un certain nombre d'instructions (boucle). Nous pouvons ainsi déléguer cette tâche à un périphérique et libérer le processeur pour effectuer d'autres opérations de calculs.

Dans cette optique l'approche consiste à programmer le Timer depuis le processeur pour lui demander de générer une interruption quand le temps à mesurer est écoulé. Cette interruption sera matérialisée par une sortie du Timer qui changera d'état (un signal qui passera de '0' à '1'). Un Timer offre typiquement la possibilité de configurer plusieurs paramètres. Un exemple est la durée du temps à mesurer, qui est exprimée en nombre de cycles d'horloge. Un autre exemple est le mode de fonctionnement qui peut être défini comme diviseur ou monostable.

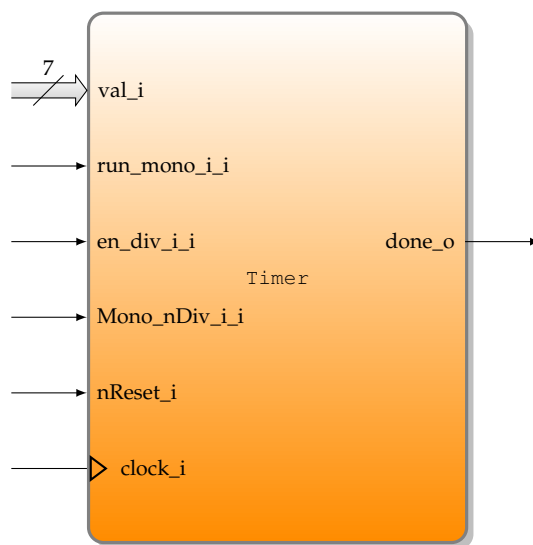
S'il y a besoin de mesurer un retard une seule fois nous pouvons utiliser le mode monostable. Le retard est mesuré une fois, une interruption annonce au microprocesseur que le temps s'est écoulé et le Timer attend une nouvelle requête.

Si le processeur doit traiter une tâche périodiquement nous pouvons utiliser un Timer en mode diviseur. Dans ce mode le Timer générera une interruption périodiquement afin de relancer la tâche au niveau du processeur.

Objectif

Le but de ce projet est de concevoir, réaliser et tester une description VHDL synthétisable d'un circuit de temporisation ("Timer" en anglais) décrit ci-après.

Symbole du Timer

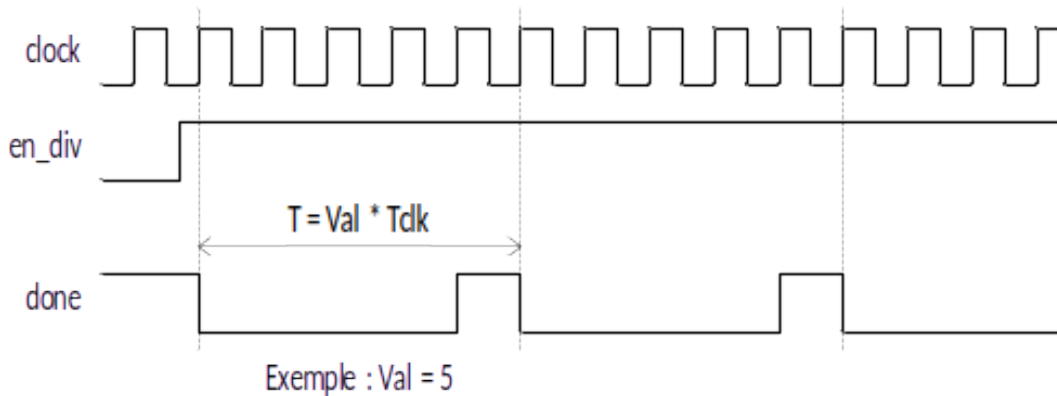


Nom	Dir.	Taille	Description
clk_i	in	1	Signal d'horloge synchrone
nReset_i	in	1	Signal de réinitialisation asynchrone
Mono_nDiv_i_i	in	1	Signal de démarrage
en_div_i_i	in	1	Signal d'activation du mode diviseur
run_mono_i_i	in	1	Signal d'activation du mode monostable
val_i	in	7	Entrée indiquant la période du signal de sortie Done en mode diviseur, ou durée du délai en mode monostable Val varie de 2 à 127
done_o	out	1	Sortie du Timer se comportant selon le mode sélectionné.

Description des modes de fonctionnement du Timer

Mode diviseur :

Lorsque l'entrée Mono_nDiv est inactive (état '0') et que en_div est actif, le Timer est en mode diviseur de fréquence. Le signal done a une période de $Val * T_{clk}$.



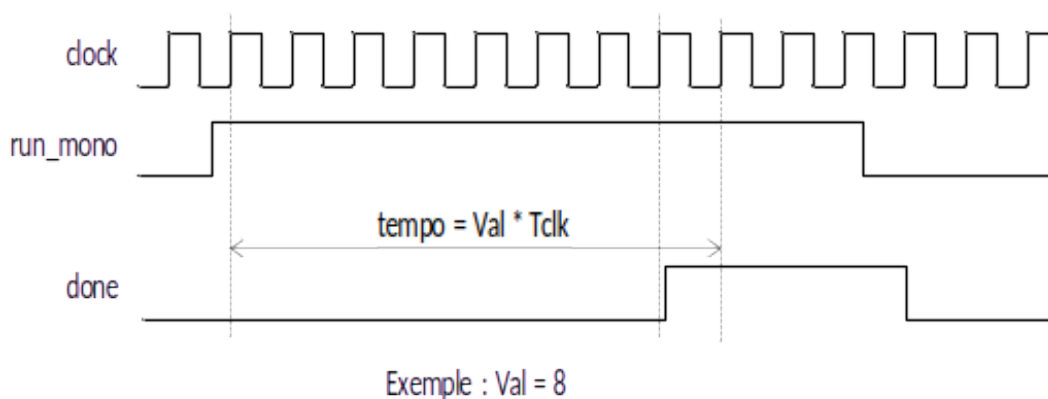
Lorsque le signal en_div est inactif, le signal done est à l'état '1' au prochain flanc montant du clock.

Mode monostable :

Lorsque l'entrée Mono_nDiv est active, le Timer est en mode monostable. Le monostable est activé à l'aide de l'entrée run_mono.

Le délai d'attente du monostable est enclenché au flanc montant du clock qui suit l'activation du signal run_mono. Le signal done doit être actif (fin du délai) lors du flanc montant du clock ayant lieu après le délai de $Val * T_{clk}$. Le chronogramme ci-dessous montre le timing du signal done.

Si l'entrée run_mono est réactivée alors que la tempo n'est pas encore écoulée, soit done encore inactif, alors le monostable est réarmé.



Lorsque le signal run_mono est inactif, le signal done est à l'état '0' au prochain flanc montant du clock.

Documents à rendre

Pour ce laboratoire, vous travaillerez par groupe de 2.

Vous devrez rendre à l'issu de ce travail de laboratoire 2 fichiers, soit :

- **Un fichier PDF** avec votre rapport de laboratoire comprenant l'ensemble de vos explications, étapes de conception, schémas (info : scanner schémas à la main), les preuves des vérifications réalisées, réponses aux différentes étapes et questions demandées, ainsi que les descriptions VHDL synthétisables
- **Une archive zip ou tar.gz** avec vos répertoires *src/...* et *src_tb/...* ainsi que vos scripts tcl

Vous devez déposer votre archive sur la page **26_HEIG-VD_SysLog2 / 26_HEIG-VD_CSN** du site Cyberlearn de la HES-SO (<https://cyberlearn.hes-so.ch/>)

Marche à suivre

1. Analyser le fonctionnement du circuit Timer afin d'identifier des fonctionnalités de base séquentielles et/ou combinatoires.
2. Concevoir une décomposition fonctionnelle du circuit Timer en visant une quantité de matériel minimum. Vous devez donner la table de fonctionnement synchrone de votre solution puis établir le schéma bloc selon la décomposition d'un système séquentiel simple vue en cours.
3. Décrire le circuit Timer à l'aide d'une description VHDL synthétisable.
4. Tester la description VHDL du circuit Timer à l'aide d'une simulation manuelle interactive avec `console_sim.vhd` et la console REDS.
5. Lorsque vous avez vérifié manuellement votre Timer, vous pouvez présenter vos résultats de simulation au professeur ou à l'assistant. Il pourra alors fournir le fichier d'un banc de test automatique

Vous devrez alors réaliser une simulation automatique de votre design

6. Faire la synthèse et le placement routage de votre solution pour le Timer à l'aide du logiciel Quartus. Vous utiliserez le top spécifique pour la carte Max-V précédemment complété : `maxv_top.vhd`. Voir dans ce fichier les I/O utilisés pour les signaux du Timer.

La carte Max-V comprend un CPLD de type : 5M570ZF256C5

Faire l'assignation des pins pour la carte Max-V 80-25p dans Quartus II, soit :

- aller dans le menu : Assignement → Import Assignments. . .
- sélectionner le fichier `maxv_top_assignment.qsf` dans le répertoire `src_cp1d`

7. Intégrer votre solution en programmant une maquette MaxV 80p-25p avec le fichier `*.pof` généré lors du placement et routage.
8. Tester le circuit Timer sur une carte Max-V. Faites des mesures à l'aide de l'oscilloscope afin de vérifier le fonctionnement.

Question :

Indiquez quel est le comportement de votre solution dans le cas où Val vaut 0 et 1. Vous donnerez une explication théorique et une preuve par la simulation manuelle.